

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
```

```
df = pd.read_csv("Diwali Sales Data.csv", encoding='latin1')
df
```

	User_ID	Cust_name	Product_ID	Gender	Age Group	Age	Marital_Status		State	Zone	Occupation	Product_Cat
0	1002903	Sanskriti	P00125942	F	26-35	28		0	Maharashtra	Western	Healthcare	
1	1000732	Kartik	P00110942	F	26-35	35		1	Andhra Pradesh	Southern	Govt	
2	1001990	Bindu	P00118542	F	26-35	35		1	Uttar Pradesh	Central	Automobile	
3	1001425	Sudevi	P00237842	M	0-17	16		0	Karnataka	Southern	Construction	
4	1000588	Joni	P00057942	M	26-35	28		1	Gujarat	Western	Food Processing	
...	...	...	...	...	...	...		...	...	...	...	
11246	1000695	Manning	P00296942	M	18-25	19		1	Maharashtra	Western	Chemical	
11247	1004089	Reichenbach	P00171342	M	26-35	33		0	Haryana	Northern	Healthcare	V
11248	1001209	Oshin	P00201342	F	36-45	40		0	Madhya Pradesh	Central	Textile	
11249	1004023	Noonan	P00059442	M	36-45	37		0	Karnataka	Southern	Agriculture	
11250	1002744	Brumley	P00281742	F	18-25	19		0	Maharashtra	Western	Healthcare	

11251 rows × 15 columns

Next steps: [Generate code with df](#) [New interactive sheet](#)

```
df.duplicated().sum()
```

```
np.int64(8)
```

```
df = df.drop_duplicates()
```

```
df.shape
```

```
(11243, 15)
```

```
df.drop(['Status', 'unnamed1'], axis = 1, inplace = True)
```

/tmp/ipykernel_2614/1289525744.py:1: SettingWithCopyWarning:
A value is trying to be set on a copy of a slice from a DataFrame

See the caveats in the documentation: https://pandas.pydata.org/pandas-docs/stable/user_guide/indexing.html#returning
df.drop(['Status', 'unnamed1'], axis = 1, inplace = True)

```
df.isnull().sum()
```

	0
User_ID	0
Cust_name	0
Product_ID	0
Gender	0
Age Group	0
Age	0
Marital_Status	0
State	0
Zone	0
Occupation	0
Product_Category	0
Orders	0
Amount	12

dtype: int64

df.dtypes

	0
User_ID	int64
Cust_name	object
Product_ID	object
Gender	object
Age Group	object
Age	int64
Marital_Status	int64
State	object
Zone	object
Occupation	object
Product_Category	object
Orders	int64
Amount	float64

dtype: object

df['Amount'].median()

8109.0

df['Amount'].fillna(df['Amount'].median(), inplace = True)

/tmp/ipykernel_2614/1184143839.py:1: FutureWarning: A value is trying to be set on a copy of a DataFrame or Series th
The behavior will change in pandas 3.0. This inplace method will never work because the intermediate object on which
For example, when doing 'df[col].method(value, inplace=True)', try using 'df.method({col: value}, inplace=True)' or d

```
df['Amount'].fillna(df['Amount'].median(), inplace = True)
/tmp/ipykernel_2614/1184143839.py:1: SettingWithCopyWarning:
A value is trying to be set on a copy of a slice from a DataFrame
```

See the caveats in the documentation: https://pandas.pydata.org/pandas-docs/stable/user_guide/indexing.html#returning
df['Amount'].fillna(df['Amount'].median(), inplace = True)

df.isnull().sum()

	0
User_ID	0
Cust_name	0
Product_ID	0
Gender	0
Age Group	0
Age	0
Marital_Status	0
State	0
Zone	0
Occupation	0
Product_Category	0
Orders	0
Amount	0

dtype: int64

df.columns

Index(['User_ID', 'Cust_name', 'Product_ID', 'Gender', 'Age Group', 'Age', 'Marital_Status', 'State', 'Zone', 'Occupation', 'Product_Category', 'Orders', 'Amount'], dtype='object')

df['User_ID'].value_counts()

	count
User_ID	
1001680	24
1003808	23
1001941	22
1004425	20
1002665	19
...	...
1003114	1
1004135	1
1002111	1
1003964	1
1005735	1

3755 rows × 1 columns

dtype: int64

df[df['User_ID'] == 1001680]

	User_ID	Cust_name	Product_ID	Gender	Age Group	Age	Marital_Status		State	Zone	Occupation	Product_Ca
13	1001680	Vasudev	P00324942	M	26-35	26	1	Andhra Pradesh	Southern		Automobile	
602	1001680	Vasudev	P00064342	M	26-35	26	0	Madhya Pradesh	Central		Food Processing	
779	1001680	Vasudev	P00036042	M	26-35	26	0	Gujarat	Western		Healthcare	
793	1001680	Vasudev	P00116142	M	26-35	26	0	Uttar Pradesh	Central		Aviation	
934	1001680	Vasudev	P00047742	M	26-35	26	1	Gujarat	Western		Banking	
1557	1001680	Vasudev	P00199742	M	26-35	26	0	Karnataka	Southern		Healthcare	Footwear
1666	1001680	Vasudev	P00179342	M	26-35	26	1	Uttar Pradesh	Central		Media	
1695	1001680	Vasudev	P00122942	M	26-35	26	0	Maharashtra	Western		Govt	
2216	1001680	Vasudev	P00067842	M	26-35	26	1	Uttar Pradesh	Central		Banking	
2444	1001680	Vasudev	P00006942	M	26-35	26	1	Delhi	Central		Hospitality	Hand & Pow
3127	1001680	Vasudev	P00339342	M	26-35	26	1	Madhya Pradesh	Central		IT Sector	
4531	1001680	Vasudev	P00046242	M	26-35	26	0	Uttarakhand	Central		Aviation	Elect (
4797	1001680	Vasudev	P00184442	M	26-35	26	0	Uttarakhand	Central		Hospitality	Clothing &
5058	1001680	Vasudev	P00164042	M	26-35	26	0	Maharashtra	Western		Automobile	Clothing &
5338	1001680	Vasudev	P00254942	M	26-35	26	1	Uttar Pradesh	Central		Healthcare	Sports P
6248	1001680	Vasudev	P00021442	M	26-35	26	1	Uttar Pradesh	Central		Banking	
6405	1001680	Vasudev	P00004342	M	26-35	26	0	Maharashtra	Western		Govt	Elect (
6453	1001680	Vasudev	P00049642	M	26-35	26	0	Andhra Pradesh	Southern		IT Sector	Elect (
7332	1001680	Vasudev	P00021142	M	26-35	26	0	Gujarat	Western		Healthcare	Clothing &
7418	1001680	Vasudev	P00252942	M	26-35	26	0	Jharkhand	Eastern		Chemical	Clothing &
8819	1001680	Vasudev	P00062142	M	26-35	26	1	Kerala	Southern		Textile	Clothing &
9249	1001680	Vasudev	P00016642	M	26-35	26	0	Uttar Pradesh	Central		Food Processing	
9413	1001680	Vasudev	P00214542	M	26-35	26	0	Himachal Pradesh	Northern		Hospitality	Elect (
10352	1001680	Vasudev	P00179942	M	26-35	26	1	Karnataka	Southern		IT Sector	Househo

df.columns

```
Index(['User_ID', 'Cust_name', 'Product_ID', 'Gender', 'Age Group', 'Age',  
      'Marital_Status', 'State', 'Zone', 'Occupation', 'Product_Category',  
      'Orders', 'Amount'],  
      dtype='object')
```

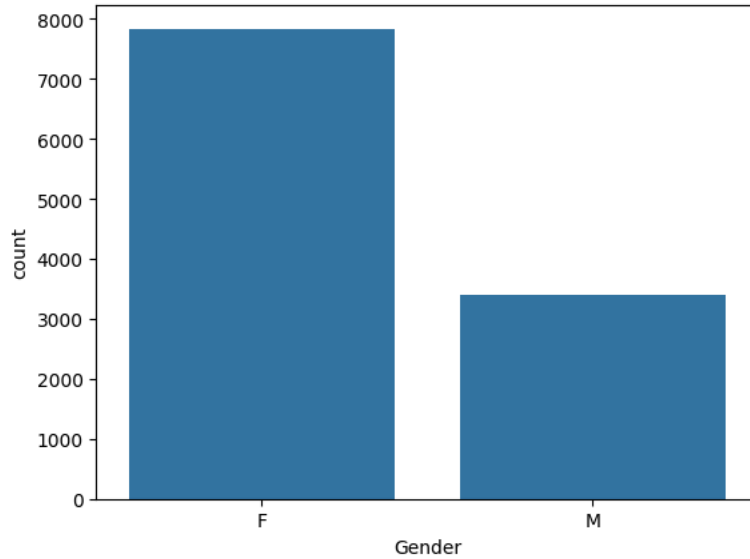
EXPLORATORY DATA ANALYSIS

```
import seaborn as sns
```

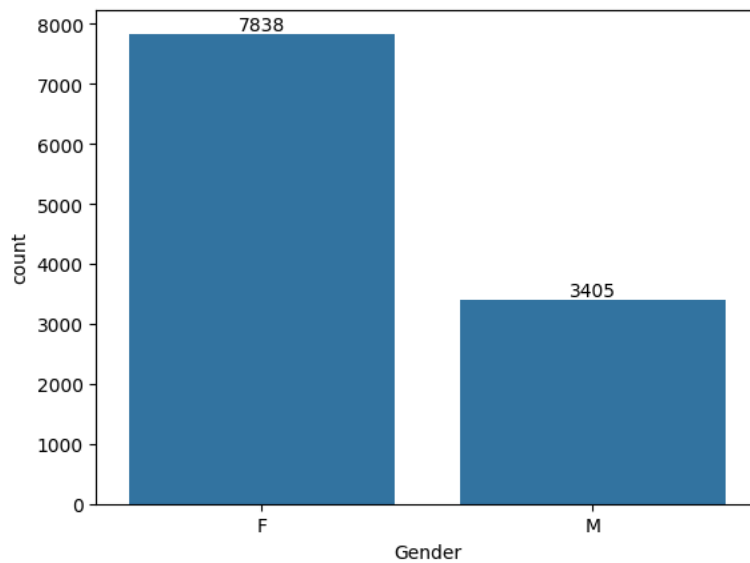
GENDER COMPARISON

```
sns.countplot(x = 'Gender', data = df)
```

<Axes: xlabel='Gender', ylabel='count'>



```
ax = sns.countplot(x = 'Gender', data = df)
for bars in ax.containers:
    ax.bar_label(bars)
```

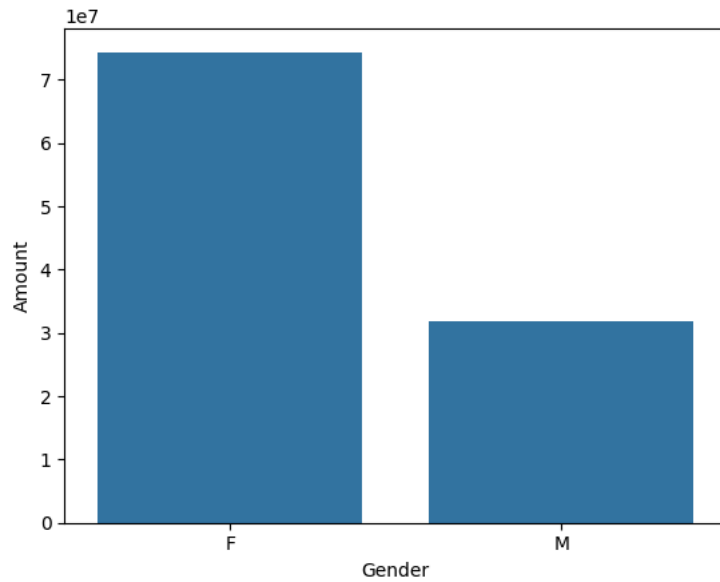


```
df.groupby(['Gender'], as_index = False)['Amount'].sum().sort_values(by = 'Amount', ascending = False)
```

	Gender	Amount	
0	F	74388772.43	
1	M	31887364.00	

```
sales_gen = df.groupby(['Gender'], as_index = False)['Amount'].sum().sort_values(by = 'Amount', ascending = False)
sns.barplot(x = 'Gender', y = 'Amount', data = sales_gen)
```

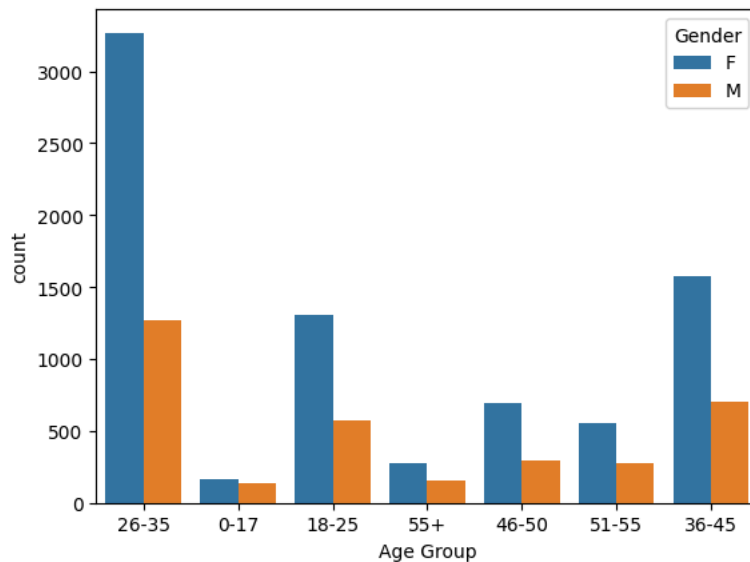
<Axes: xlabel='Gender', ylabel='Amount'>



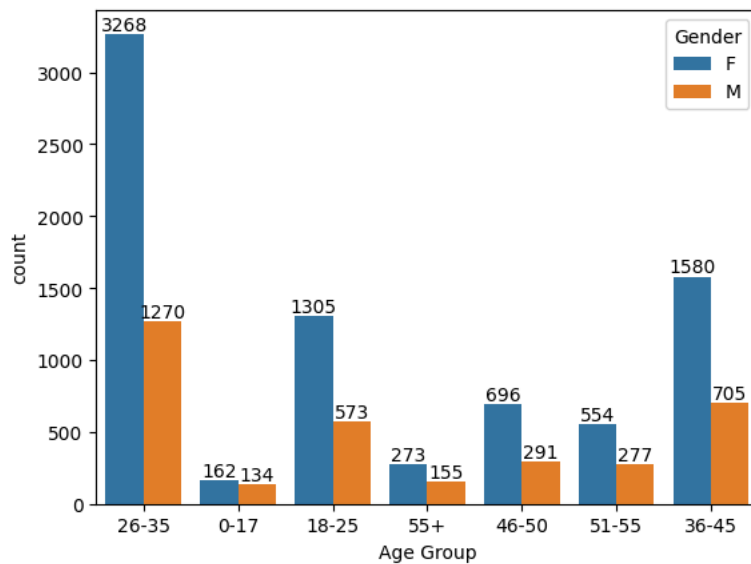
✓ AGE GROUP ANALYSIS

```
sns.countplot(data = df, x = 'Age Group', hue = 'Gender')
```

<Axes: xlabel='Age Group', ylabel='count'>



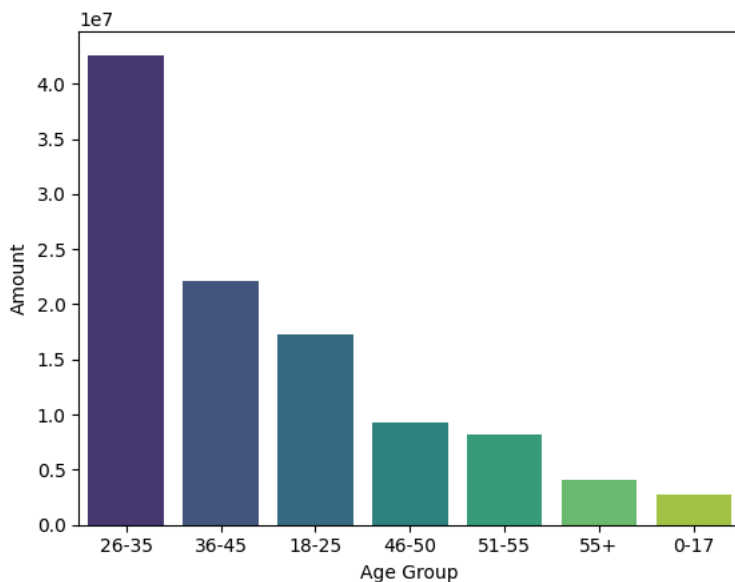
```
ax = sns.countplot(data = df, x = 'Age Group', hue = 'Gender')
for bars in ax.containers:
    ax.bar_label(bars)
```



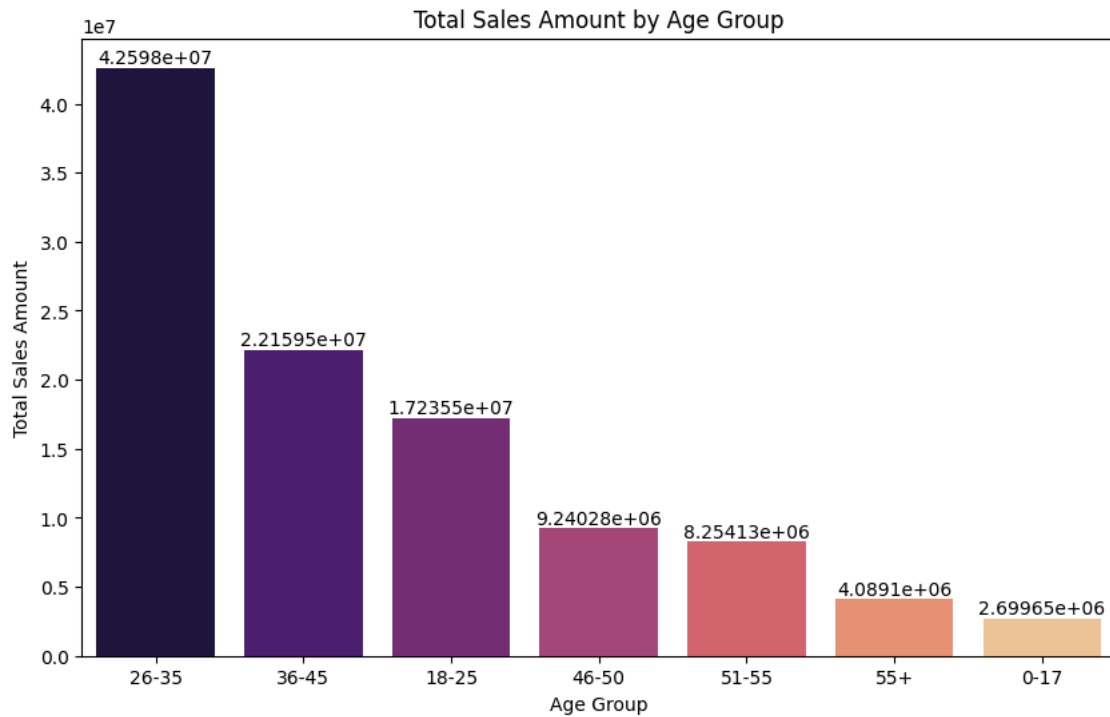
```
df.columns
```

```
Index(['User_ID', 'Cust_name', 'Product_ID', 'Gender', 'Age Group', 'Age',
      'Marital_Status', 'State', 'Zone', 'Occupation', 'Product_Category',
      'Orders', 'Amount'],
      dtype='object')
```

```
sales_age = df.groupby(['Age Group'], as_index = False)['Amount'].sum().sort_values(by = 'Amount', ascending = False)
sns.barplot(x = 'Age Group', y = 'Amount', data = sales_age, hue = 'Age Group', palette = 'viridis', legend=False)
plt.show()
```



```
plt.figure(figsize=(10, 6))
ax = sns.barplot(x = 'Age Group', y = 'Amount', data = sales_age, hue = 'Age Group', palette='magma', legend=False)
for bars in ax.containers:
    ax.bar_label(bars)
plt.title('Total Sales Amount by Age Group')
plt.xlabel('Age Group')
plt.ylabel('Total Sales Amount')
plt.show()
```



- From above graphs we can say that most of the buyers are of age 26-35 where female are the ones buying the most.

```
df.columns
```

```
Index(['User_ID', 'Cust_name', 'Product_ID', 'Gender', 'Age Group', 'Age',  
      'Marital_Status', 'State', 'Zone', 'Occupation', 'Product_Category',  
      'Orders', 'Amount'],  
      dtype='object')
```

STATE-WISE ANALYSIS

```
sales_state = df.groupby('State', as_index = False)['Orders'].sum().sort_values(by = 'Orders', ascending = False)  
sales_state
```

	State	Orders	
14	Uttar Pradesh	4809	
10	Maharashtra	3799	
7	Karnataka	3241	
2	Delhi	2744	
9	Madhya Pradesh	2259	
0	Andhra Pradesh	2054	
5	Himachal Pradesh	1568	
8	Kerala	1137	
4	Haryana	1106	
3	Gujarat	1067	
1	Bihar	1058	
6	Jharkhand	953	
15	Uttarakhand	824	
12	Rajasthan	555	
11	Punjab	495	
13	Telangana	312	

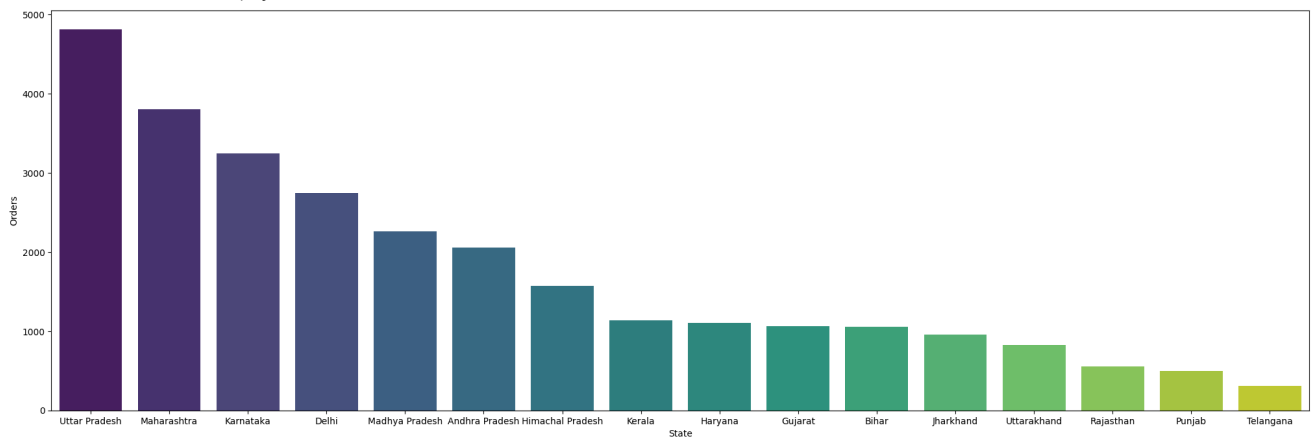
Next steps: [Generate code with sales_state](#) [New interactive sheet](#)

```
plt.figure(figsize = (25,8))
order_state = df.groupby('State', as_index = False)['Orders'].sum().sort_values(by = 'Orders', ascending = False)
sns.barplot(x = 'State', y = 'Orders', data = order_state, palette = 'viridis')
```

/tmp/ipykernel_2614/76822437.py:3: FutureWarning:

Passing `palette` without assigning `hue` is deprecated and will be removed in v0.14.0. Assign the `x` variable to `hue`

```
sns.barplot(x = 'State', y = 'Orders', data = order_state, palette = 'viridis')
<Axes: xlabel='State', ylabel='Orders'>
```

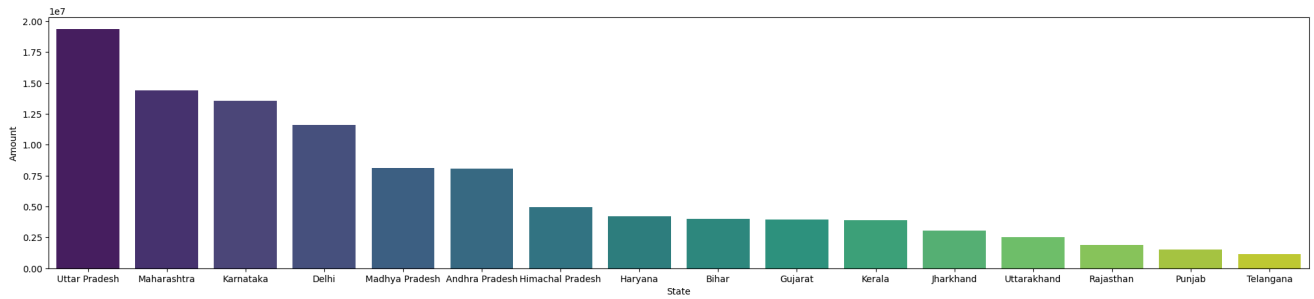


```
plt.figure(figsize = (25, 5))
sales_state = df.groupby('State', as_index = False)['Amount'].sum().sort_values(by = 'Amount', ascending = False)
sns.barplot(x = 'State', y = 'Amount', data = sales_state, palette = 'viridis')
```

```
/tmp/ipykernel_2614/3585776790.py:3: FutureWarning:
```

Passing `palette` without assigning `hue` is deprecated and will be removed in v0.14.0. Assign the `x` variable to `h`

```
sns.barplot(x = 'State', y = 'Amount', data = sales_state, palette = 'viridis')
<Axes: xlabel='State', ylabel='Amount'>
```



From the above graphs we interpret that people from Uttar Pradesh, Maharashtra & Karnataka are buying the most.

Start coding or [generate](#) with AI.

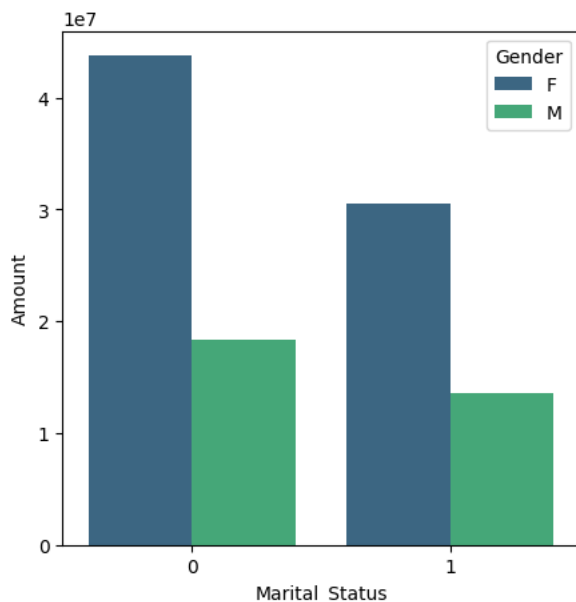
▼ MARITAL STATUS ANALYSIS

```
df.columns
```

```
Index(['User_ID', 'Cust_name', 'Product_ID', 'Gender', 'Age Group', 'Age',
      'Marital_Status', 'State', 'Zone', 'Occupation', 'Product_Category',
      'Orders', 'Amount'],
      dtype='object')
```

```
plt.figure(figsize = (5,5))
sales_marital = df.groupby(['Marital_Status','Gender'], as_index = False)['Amount'].sum().sort_values(by = 'Amount',
sns.barplot(x = 'Marital_Status', y = 'Amount', hue = 'Gender' ,data = sales_marital, palette = 'viridis')
```

```
<Axes: xlabel='Marital_Status', ylabel='Amount'>
```



▼ OCCUPATION

```
df.columns
```

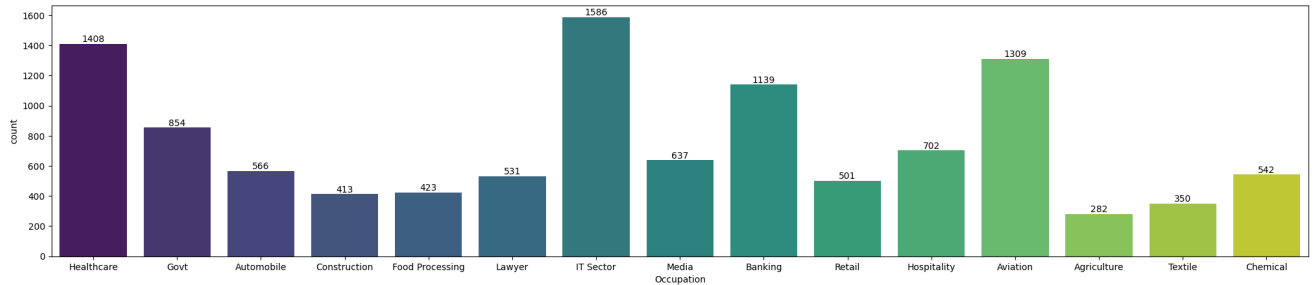
```
Index(['User_ID', 'Cust_name', 'Product_ID', 'Gender', 'Age Group', 'Age',  
      'Marital_Status', 'State', 'Zone', 'Occupation', 'Product_Category',  
      'Orders', 'Amount'],  
      dtype='object')
```

```
plt.figure(figsize = (25, 5))  
ax = sns.countplot(x = 'Occupation', data = df, palette = 'viridis')  
for bars in ax.containers:  
    ax.bar_label(bars)
```

/tmp/ipykernel_2614/2968577130.py:2: FutureWarning:

Passing `palette` without assigning `hue` is deprecated and will be removed in v0.14.0. Assign the `x` variable to `hue` to silence this warning

```
ax = sns.countplot(x = 'Occupation', data = df, palette = 'viridis')
```

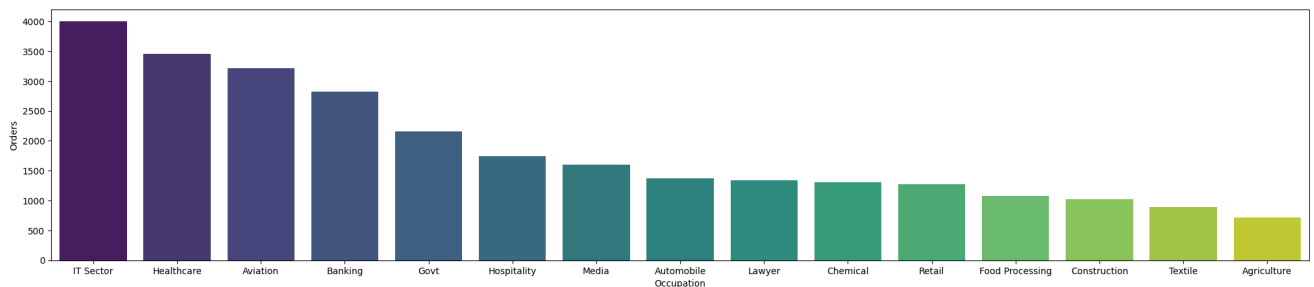


```
plt.figure(figsize = (25, 5))  
order_occupation = df.groupby('Occupation', as_index = False)['Orders'].sum().sort_values(by = 'Orders', ascending =  
sns.barplot(x = 'Occupation', y = 'Orders', data = order_occupation, palette = 'viridis')
```

/tmp/ipykernel_2614/633029122.py:3: FutureWarning:

Passing `palette` without assigning `hue` is deprecated and will be removed in v0.14.0. Assign the `x` variable to `hue` to silence this warning

```
sns.barplot(x = 'Occupation', y = 'Orders', data = order_occupation, palette = 'viridis')  
<Axes: xlabel='Occupation', ylabel='Orders'>
```



```
plt.figure(figsize = (25, 5))  
sales_occupation = df.groupby('Occupation', as_index = False)['Amount'].sum().sort_values(by = 'Amount', ascending =  
sns.barplot(x = 'Occupation', y = 'Amount', data = sales_occupation, palette = 'viridis')
```

```
/tmp/ipykernel_2614/1937696195.py:3: FutureWarning:
```

```
Passing `palette` without assigning `hue` is deprecated and will be removed in v0.14.0. Assign the `x` variable to `h
```